

Record once with a model. Replay forever without one.

A computer-use automation system for a legacy core-banking web app. An LLM drives the browser exactly one time, to record a reviewable capability artifact. Every invocation after that re-executes that artifact deterministically — no model, no tokens, no non-determinism in the path that touches money.

DISCOVERY · WITH LLM

100 s

median wall time · 13 tool calls · ~71k
tokens · 47 runs measured

REPLAY · NO LLM

7 s

median wall time · 0 tool calls · 0 tokens · 74
runs measured

RUNNING ORDER

- 01 **Open with the thesis** 2 min
- 02 **Live demo** 8 min
- 03 **The artifact** 5 min
- 04 **Discovery loop** 8 min
- 05 **Replay + outcomes** 7 min
- 06 **Human-in-the-loop** 5 min
- 07 **Isolation & evidence** 5 min
- 08 **Chat surface** 3 min
- 09 **Metrics & gaps** 4 min

REFERENCE

- The mock-app question
- Every backend file
- Patterns & keywords
- Questions to expect

On your proposed order

Your instinct is right on the two endpoints and wrong in the middle.

Opening on the live demo is correct — it buys the attention you then spend on architecture. Ending on metrics is correct. Three changes move this from "good engineering walkthrough" to "this person has shipped into a regulated environment":

1. **Do not call Docker and Postgres "cosmetic."** A per-run ephemeral container and an append-only, redacted evidence store *are* the compliance story. In a bank they aren't garnish — they're the reason the thing is deployable at all. Move them later (after the engines) and reframe as "*how this survives a security review.*" Calling them cosmetic invites the CTO to discount them, and they're some of your strongest material.
2. **Put the artifact second, before discovery.** Everything in your system either *produces* an artifact or *consumes* one. Show the noun before the verbs and both engines explain themselves in half the time. Right now discovery lands before the audience knows what it's building.
3. **Don't defer edge cases to their own segment.** In a regulated domain the CTO is thinking about failure the entire time — a dedicated late slot reads as an afterthought. Fold the outcome taxonomy into replay, where it lives in the code, and spend the freed slot on *known gaps and what I'd do next*, which is the segment that actually demonstrates seniority.

One structural addition: give the talk a spine by naming the thesis in the first sixty seconds and hanging every segment off it. The record-once / replay-free split is the genuinely interesting idea here. Say it before the demo, not after.

01 Open with the thesis

2 min • no slides

Before anything runs. This is the sentence the rest of the talk is evidence for.

SAY ROUGHLY THIS

"A general computer-use agent is the wrong shape for core banking. It's slow, it costs tokens on every run, and it can make a different decision on Tuesday than it made on Monday. You can't put that in front of a ledger.

So I split it. The model runs exactly once, to record — it explores the flow and compiles a reviewable artifact. Every invocation afterwards replays that artifact deterministically, with no model in the loop at all. Recording takes a hundred seconds and seventy thousand tokens. Replaying takes seven seconds and zero. Same flow, same result, and the second one is auditable."

Then name the three consequences, because they're your next three segments: the artifact is a **reviewable, versioned, approvable object**; the replay path is **deterministic enough to classify failures precisely**; and the expensive, non-deterministic part is **confined to a recording session a human signs off on**.

02 Live demo

8 min

Chat → proposal → confirmation → live browser → escalation → resume → structured result.

Drive it from the chat surface with the noVNC feed visible. The beat you want is **the escalation** — that's the moment the room understands this isn't a demo toy. Pick a capability with a risky commit step (`meridian_transfer_funds` or `meridian_open_new_share`) so the run genuinely stops and waits for you.

The sequence to narrate

1. Ask in plain language. The agent matches against the catalog of *already-approved* capabilities — it can only propose what a human has signed off.
2. It returns a proposal with the exact parameters. **Nothing runs on a proposal.** Note that the confirmation gate is parsed deterministically before the model is ever asked to interpret the reply.
3. Confirm. A real browser starts in a container; the feed appears.
4. It walks the flow and **stops at the commit step.** Show the escalation banner: run id, step index, screenshot, reason.
5. Take control in the same live session — not a fresh one. Do the step yourself.
6. Resume. It picks up, verifies the success checkpoint, returns a structured result with a real confirmation number.
7. Open the run's event log. Every decision, action and screenshot, redacted, in order.

THE LINE TO LAND WHILE IT'S PAUSED

"It stopped here because this step was classified risky when it was recorded. And the important part is that the human takes over the same browser session the agent was driving — not a copy, not a re-login. Whatever I do here, the agent picks up from the state I leave behind. If I finish the whole flow myself, it notices and reports success rather than trying to run it a second time."

KEEP IN YOUR POCKET

That last clause is a real guard, not a hypothetical. A transfer had already posted — confirmation number in hand — and a completeness check pushed the agent back for a missing memo field. It returned to the member record, reopened Funds Transfer, and started selecting shares for a *second* transfer. It stopped only because it ran out of steps.

The fix is in `discovery/agent.py` : once any step classified risky has executed, `finish` is final and no quality gate may send the loop around again. *"An incomplete recording is a bad artifact. A duplicated financial transaction is a different category of problem."*

03 The artifact

5 min

The contract. Show the actual JSON on screen.

Pull up a recorded artifact and walk the fields. This is the highest-leverage screen in the talk: it makes both engines self-explanatory, and it's the thing a bank's change-management process would actually review.

<code>schemas.py</code> → Artifact	Capability name, version, target host, description, input schema, output schema, ordered steps, success checkpoint, outcome patterns, approval status.
<code>schemas.py</code> → Step	One UI action: type, locator, value <i>or</i> a parameter binding, extraction name, risk tier, optional checkpoint.
<code>schemas.py</code> → Locator	Role / text / CSS strategy, plus an optional <code>fallback</code> chain — a second way to find the same control when the first stops resolving.
<code>schemas.py</code> → ValueSource	Deliberately binary. Absent means "literal, baked in." Present means "replay substitutes the caller's parameter here." There is no third state.

schemas.py → Checkpoint

Element visible / text present / URL matches. How a step or a whole run proves it actually worked.

schemas.py → OutcomePattern

A page state that is neither success nor a bug — "insufficient funds", "no such member" — with its retry budget and optional recovery action.

Four points to make

- **It's diffable.** A capability is a JSON document. Two versions compare in a pull request. That's change management for automation, in a vocabulary a bank already has.
- **It has a lifecycle.** `draft → approved → retired`, with `rejected` as a distinct terminal state. A retired version can be re-approved — that's your rollback path. A rejected one cannot. Before those were separate states, rollback was a one-way door.
- **Discovery-time secrets never land in it.** A parameter's literal value from the recording session is kept aside and surfaced as an *example* for a reviewer — and suppressed entirely for anything credential-shaped. A real password typed during recording has no business in a JSON file on disk.
- **Risk is recorded, not inferred at runtime.** Each step carries its tier from the moment it was recorded and reviewed. Replay doesn't re-decide what's dangerous.

schema-first design

Pydantic validation

versioned contract

DTO / value object

approval workflow

immutable artifact

04 The discovery loop

8 min

How an artifact gets made — and the eight guards that stop it making a bad one.

One file: `comp_use/discovery/agent.py`. Walk the loop as a sequence, then spend the back half on the guards, because the guards are the interesting engineering.

The loop, per step

1. **Check for human intent.** Stop-run beats takeover-request; both checked before spending a token.
2. **Observe.** Read the accessibility tree, not the HTML — supplemented with form field `name / id / type` attributes, never field *values*.
3. **Tokenize.** Account numbers, transaction ids and dollar amounts are reversibly swapped for placeholders before the tree reaches the model.
4. **Decide.** One LLM call returns one JSON action. Provider usage is logged per call, so cost accrues visibly rather than as a total at the end.
5. **Validate and detokenize** at a single boundary — everything downstream sees real values.
6. **Allowlist check** on URL and action type.
7. **Classify risk** from the control's label and the current URL.
8. **Escalate if risky**, then act on the live page.
9. **Re-check the allowlist** — a click may have navigated somewhere new, and that was never validated.
0. **Check for a known fault page** *before* recording the step.
1. **Record the step** — value omitted if it's a parameter.

THE FRAMING LINE

"A naive version of this loop is about forty lines. Most of this file is not the loop — it's the guards. Every one of them is a specific way a recording went wrong against the real system, and every one produces a bad artifact that records perfectly and only fails on first real use. That's the worst possible shape for a bug, so they're all checked while the model is still around to be asked for something better."

The guards worth naming

GUARD	WHAT IT CATCHES
loop detection	Counts every (url, action, locator) signature. Three repeats escalates. Catches multi-step cycles too, since each component accumulates its own count.
dead-end counter	Three consecutive invalid or skipped decisions escalates. Distinct from loop detection, which fires on <i>valid</i> decisions that make no progress.
post-commit finish lock	Once a risky step has run, finish is final. No quality gate may re-run the flow.
missing-parameter gate	An operator-declared parameter that never became a real parameter means the flow never entered that value — the capability would submit a pre-filled form unchanged and report success.
no-extract gate	A read-only capability with an empty output schema returns nothing to its caller. Asked once, never enforced — some goals genuinely have nothing to report.
unusable-extract check	Rejects a locator anchored on the value it just read, or one grabbing the whole page. Uses a length ratio, so a short leading label still passes.
dropdown completeness	Warns if a combobox was ever visible but no selection was performed — a field may have silently shipped at its default.
fault detection	Recognises the host's own error pages before recording, so an improvised recovery doesn't get baked in as canonical steps.

THE BEST STORY IN THIS SEGMENT

The balance-reading capability oscillated for six minutes across ten progressively more elaborate selectors — every one of which had already worked. Because every dollar amount on the page matched a redaction pattern, the model saw a table of `[[TOK...]]` and read it as a failed extract.

The fix isn't code, it's a sentence appended to the history: *the tokens mark real values that were captured and redacted; their presence means the extract worked; the rows and columns around them are the proof; you will never be shown them.* Worth telling because it's a failure that only appears when privacy controls and an agent loop meet in production.

sense-plan-act loop

ReAct

circuit breaker

critic / verifier gate

bounded retry

pseudonymization at trust boundary

allowlist over denylist

accessibility-tree grounding

05 Replay and the outcome taxonomy

7 min

Fold your "edge cases and failure modes" segment in here. This is where it lives in the code.

`comp_use/replay/engine.py` is an interpreter over the recorded step list. Open by stating the constraint plainly: **this file imports no LLM client and never will.** That's the promise the whole architecture rests on, and it's checkable in one grep.

Per step

Validate inputs against the declared schema → check for a diverged outcome page *before* attempting the step → escalate if risky → substitute parameters → allowlist → act → re-check the allowlist → verify the step checkpoint. Then verify the artifact's success checkpoint at the end.

The taxonomy — the centrepiece of this segment

OUTCOME	MEANS	RETRIED?
success	The success checkpoint is satisfied. Outputs returned.	—
business_outcome	The automation worked perfectly and the answer is no. Insufficient funds, no such member, supervisor override required, invalid email format.	Never
recoverable	A transient host condition — a maintenance interstitial, a session timeout. Has a retry budget and optionally a declared recovery action.	Up to 3
hard_failure	Nothing explains it. A locator stopped resolving, a checkpoint didn't match, or the host returned its own application error.	No
input_error	Outside the taxonomy on purpose: the request never reached the host, because the caller didn't supply what the artifact declares it needs.	No

THE DISTINCTION TO SELL HARD

"The one worth dwelling on is business_outcome. Most agent frameworks have two states — it worked, or it didn't. But 'insufficient funds' is not a failure. The automation did exactly its job and the bank correctly said no. If you report that as an incident you'll wake someone up at 3am for a customer who's overdrawn.

And it goes further than the category. The host names the actual rule that failed underneath a generic banner, so the pattern carries a locator for that line and appends it. The caller doesn't get 'transaction could not be validated' — they get 'insufficient available balance in the source share.' For a rejected request, the category is the less useful half of the answer."

Two subtle behaviours to mention

- **Partial outputs travel with every result.** A run that read three values and then died still returns those three. Dropping them made a partial answer indistinguishable from no answer.

- **A recovered-from condition downgrades a later failure.** A maintenance interstitial was correctly matched and dismissed — and the dismissal returned the host to a signed-out page, so every later step failed against a page showing no error at all. Reporting that as a hard failure blames the automation for a host condition it handled correctly, so it reports *recoverable*: re-invoke from the start. Deliberately not solved by auto-restarting from step zero, because the earlier steps may be irreversible.

THE FALSE-POSITIVE STORY

The permission-denied pattern was originally anchored on the banner text `SUPERVISOR OVERRIDE REQUIRED`. But the legitimate Place Hold *form* carries that same banner as a "restricted function" warning label. So every Place Hold replay matched it the instant it reached the form — reporting "not authorized" even when signed on as a supervisor who *was* authorized, and bailing out before the flow ran at all.

It's now anchored on the sentence that appears only on the real denial page. Good story because it shows the failure mode of pattern-matching a page, and shows you found it.

deterministic replay

interpreter pattern

typed result object

retry budget

graceful degradation

fail-closed

06 Human-in-the-loop

5 min

Four distinct control operations that are easy to conflate and are deliberately kept apart.

Both engines poll the same controller. The transport behind it is swappable: a terminal prompt for CLI runs, a thread-safe queue for HTTP-driven runs.

OPERATION	INTENT	AFTER
escalate	The engine stops itself — risky step, loop, unrecognised fault.	Human resumes; same run continues.
takeover	A human asks to drive. Honoured at the next step boundary, before a token is spent.	Same handshake, same run.
interrupt	End the run for good.	Run marked <i>interrupted</i> , not <i>failed</i> .
cancel	Unblock an escalation nobody is going to resume.	Run finishes reporting a clean failure.

Three details worth surfacing

- **Interrupt is a `BaseException` , on purpose.** Every layer between the step loop and the worker thread catches broad `except Exception` and converts what it caught into a *reported failure*. An operator stopping a run is not a failure of the capability and must not be recorded as one — subclassing `BaseException` makes that true by construction rather than by remembering to re-raise it in a dozen places. A good answer if you're asked about code quality.
- **Every handoff is evidenced as a diff.** Accessibility tree and screenshot before, the same after, plus the human's own one-line note. You can see exactly what a person changed while they had control — bounded at 200 diff lines so one handoff can't dominate the log.
- **Resume checks for success before attempting anything.** A human who takes over has no obligation to stop where the agent stopped. Confirmed live: a supervisor-override denial was resolved by the operator completing the entire Place Hold themselves. Without that check, the next recorded step runs against a page that has already moved past it and reports a hard failure for a run that succeeded.

human-in-the-loop

strategy pattern

producer-consumer queue

dual control

four-eyes principle

idempotent resume

Reframed from "cosmetic" to: how this survives a bank's security review.

This is the segment you were going to demote. Don't — it's the part a CTO in finance is obliged to care about, and you have real answers.

Isolation — one container per run

`comp_use/sandbox.py` spawns an ephemeral Docker container per run: headed Chromium under Xvfb, driven over CDP, viewable and controllable over noVNC, torn down by a context manager's `finally` — which means even an interrupt unwinding through the stack stops the container. **The container-per-run lifetime is what makes "take over the same live session" true** when the browser isn't running somewhere a human can just look at.

Say the numbers: **~450 MiB per concurrent run, 1.2 GiB worst case, no containers left behind**. Then the honest one — 187 MB and 37% CPU is the XFCE desktop, pure overhead you'd strip for production.

Evidence — append-only and redacted at the boundary

Three tables: `runs`, `run_events`, `run_screenshots`, cascading on delete. Every event is redacted *before* it is written — the store never holds an unredacted value. Points to make:

- **Every write is best-effort and non-fatal.** A Postgres outage must never crash a run driving a banking flow. Reads degrade to empty rather than raising.
- **Orphan reaping at startup.** A killed process leaves rows saying *running* forever. A fresh process owns no threads from a previous one, so anything still open at startup is by definition orphaned — no race to lose. This was five phantom runs whose browsers hadn't existed for hours.
- **Postgres is primary, not a mirror.** The filesystem layout survives only as the no-database fallback for local dev.

The three-layer data boundary

LAYER	DIRECTION	PROTECTS
allowList	—	URL prefixes and action types. Checked before <i>and</i> after every action, because a click can navigate off-domain and that was never validated by the pre-check.
redact()	one-way	Logs, console, evidence. Replaces matches with <code>[REDACTED]</code> .
tokenizer	two-way	The LLM call. The model reasons over stable placeholders; the browser always acts on real data.

Both read the same regex list from settings. Neither has any field-specific logic. Covering a new sensitive shape — SSN, phone — is one regex, picked up by both consumers, with no code change to either. Say that: it's the difference between a privacy feature and a privacy architecture.

VOLUNTEER THE GAP BEFORE THEY FIND IT

"One channel isn't covered. When the accessibility tree isn't enough to pin down a control, it falls back to sending a screenshot to a vision model — and you can't regex pixels. That's a documented exposure, and the mitigation is pointing the client at a local or on-prem model, which is a config change, not a rewrite."

Naming your own gap unprompted is worth more here than any feature you could describe instead.

- ephemeral infrastructure
- blast-radius containment
- append-only audit log
- data minimization
- defence in depth
- readiness probe
- graceful degradation

08 The chat surface

3 min

Keep it short. It's the least differentiated part of the system and you've already demoed it.

The point is architectural, not conversational: **the chat agent never imports the engines**. It's a pure client of the same HTTP surface everything else uses. It proposes; the endpoint executes through the

identical code path a direct API call takes. There is no second, weaker way to start a run.

- The model gets three tools: propose an invoke, propose a discovery, resolve a pending confirmation. It can only propose capabilities a human has already approved.
- **The confirmation gate is parsed deterministically first.** An exact *yes* or *no* resolves without asking the model at all; anything nuanced falls through to it, and a malformed or unhonoured tool call **fails closed to "amend"** — never to "confirm."
- Sessions carry their own tokenizer, so token-to-value mappings stay stable across a conversation and sensitive values never reach the provider.
- Proposals are re-validated before execution — a capability can be deleted between the proposal and the confirmation.
- A per-session lock, because two overlapping requests could otherwise both read the same pending action and both start a run.

WHY THE DETERMINISTIC PRE-PARSE EXISTS

A free-tier model was observed ignoring a forced `tool_choice` entirely. The fallback path silently cleared the pending action and looped the user back to square one *no matter how many times they said yes* — on the single most safety-critical gate in the product. Hence: parse the obvious cases in code, and let the model handle only genuine ambiguity.

tool / function calling

fail-closed default

confirm-before-execute

per-session lock

thin client over shared API

09 Metrics, then gaps

4 min

Close on what it costs and what you'd do next. The gaps segment is the one that reads as senior.

	DISCOVERY WITH LLM	REPLAY NO LLM
Runs measured	47	74
Wall time — median	100 s	7 s
Wall time — mean	117 s	17 s
Wall time — min / max	12 s / 561 s	2 s / 159 s
LLM tool calls — median	13	0
Total tokens — median	70,898	0
Total tokens — range	15,815 - 122,737	0
Container peak memory	428 MiB	428 MiB

Two things to say about the numbers rather than just showing them: **discovery time is dominated by waiting on the model, not by driving the browser** — so it scales with model latency, not flow complexity. And **the 561-second outlier was provider timeouts; the 159-second replay was a human takeover**. Naming your outliers reads as someone who actually looked at the data.

End-to-end verification

All eight capabilities replayed against the live host, not mocked: five clean successes, two escalate→confirm→resume→success cycles returning real confirmation numbers, and one clean `business_outcome` — a teller attempting a supervisor-only hold, classified correctly rather than timing out.

The gaps — say these before you're asked

- **Session-timeout recovery is architecturally incomplete.** Recovering means re-authenticating and resuming the remaining steps, which a single-UI-step recovery action can't express. It's reported honestly as *recoverable* with no automatic recovery attempted, rather than misclassified.

- **The vision fallback isn't tokenized.** Pixels can't be regexed.
- **Chat sessions are in-memory.** A restart drops them. Deliberate, matching the run manager, but it wouldn't survive production.
- **Drift repair is diagnosis-only.** A failed locator gets a proposed patch saved as a new unapproved version and a human is asked to review it. Nothing is ever auto-applied. Frame this as the right default, not a limitation — self-healing selectors that silently rewrite themselves are exactly what you don't want touching a ledger.
- **XFCE is ~187 MB of pure overhead** per container. First thing to strip.

On "it was built for the mock app and now only works on MERIDIAN"

You raised this as a weakness. **At the code level it is substantially less true than you think, and the accurate version is a strength.** I checked: across the entire backend, exactly two places name the target host.

`discovery/outcome_library.py`

A registry keyed by hostname. Returns an *empty list* for any host it doesn't recognise — so an unknown target degrades to "no known outcome patterns," which is the pre-existing behaviour, not a crash.

`chat/agent.py` — system prompt

One adjective in one sentence. Cosmetic.

Everything else that mentions the host mentions it *in a comment*, documenting which live failure motivated a general mechanism. The mechanisms themselves are host-agnostic: selecting a dropdown by label before value, matching rendered text rather than HTML source, supplementing the accessibility tree with form field attributes, wildcarding numeric URL segments, the locator fallback chain.

HOW TO SAY IT — VOLUNTEER IT, DON'T WAIT TO BE ASKED

"I want to be straight about the trajectory. I built this against a mock app I controlled, then pointed it at a real legacy system, and it broke in about fifteen specific ways. The mock was too clean — real labels, real form semantics, no error taxonomy. MERIDIAN has table-based layouts with no label association, banner text that collides across pages, and dropdowns with balances baked into the option labels.

Every one of those breakages is now a named guard with a comment explaining the failure that caused it. And the host-specific part is one registry file — onboarding a new core system means adding its error pages to that registry, not touching either engine. So the honest claim isn't 'this is general' — it's 'this got general by contact with one genuinely ugly system, and the seam where the next one plugs in is a single file.'"

That framing does three things at once: it's honest, it shows you know the difference between abstraction and generality, and it answers "what would it take to point this at *our* system" before the CTO has to ask — which is the question they're actually there to have answered.

Verified before you present: the full suite is 368 passing. But note the mock-app tests exercise the Flask routes directly, not an agent run against them — so they don't prove end-to-end discovery still works there. If you plan to demo against the mock at all, check it first; if it's broken, say the mock drifted out of support once the real target became the reference implementation, which is a defensible call.

Every backend file

~4,500 lines of backend, 368 tests. In rough dependency order.

<code>comp_use/schemas.py</code>	208 lines. The contract. Every Pydantic model in the system: Artifact, Step, Locator, Checkpoint, ValueSource, OutcomePattern, ReplayResult, OutcomeType, InterventionRequest. If you read one file, read this.
<code>comp_use/config.py</code>	Environment-driven settings. Single source of truth for the URL allowlist, allowed action types, redaction regexes, model providers, step budget.
<code>comp_use/guardrail.py</code>	The policy enforcement point. Allowlist check, risky-step confirmation, one-way redaction, and the credential-shaped-parameter test that keeps passwords out of artifacts.
<code>comp_use/tokenizer.py</code>	Reversible tokenization. Stable placeholders per run, so the model can still recognise "I've seen this account before" without ever seeing the account.
<code>comp_use/surface.py</code>	The browser abstraction. An abstract <code>Surface</code> plus a Playwright implementation. Holds the locator fallback chain, robust dropdown selection, rendered-text matching and safe screenshot capture. A desktop or native surface would implement the same five methods.
<code>comp_use/evidence.py</code>	Redact-then-write logger. Postgres when configured, JSONL plus PNGs otherwise.
<code>comp_use/pg/store.py</code> <code>comp_use/pg/schema.sql</code>	Sync psycopg2 with a threaded connection pool — sync because the whole engine is synchronous and an async driver would need every call site wrapped. Every write non-fatal. Three cascading tables plus artifact storage.
<code>comp_use/llm_client.py</code>	Abstract client, an OpenAI-compatible base, two provider subclasses, a fallback wrapper that tries the second provider when the first fails, and a fake for tests. Also holds the ~130-line discovery system prompt — worth showing, since most guards have a prompt-side counterpart.
<code>comp_use/discovery/agent.py</code>	849 lines. The recording loop and its eight guards.

<code>comp_use/discovery/compiler.py</code>	Trace → Artifact. Derives the input schema from parameter bindings and the output schema from extractions; collapses adjacent duplicate steps and drops superseded extracts (output-preserving by construction — only the last write per name ever survived).
<code>comp_use/discovery/outcome_library.py</code>	Host-keyed registry of known error and business states. Resolved live at replay time rather than snapshotted into artifacts, so fixing a shared pattern reaches every already-recorded capability.
<code>comp_use/replay/engine.py</code>	593 lines. The deterministic executor. Type-checks inputs, substitutes parameters, runs steps, matches outcome patterns, manages retry budgets, resolves derived outputs by arithmetic. Imports no LLM.
<code>comp_use/escalation/controller.py</code> <code>comp_use/escalation/transport.py</code>	Human handoff. The controller evidences before/after state as a bounded tree diff; the transport swaps between a terminal prompt and a thread-safe queue, and defines the four control operations.
<code>comp_use/sandbox.py</code>	One Docker container per run. Publishes ports, then polls the CDP endpoint until Chromium actually answers — a published port only means Docker is forwarding it.
<code>comp_use/drift.py</code>	Post-mortem locator repair. Runs strictly <i>after</i> a replay has produced its final LLM-free result, so the determinism guarantee holds. Proposes a patched version; never saves, applies or replays it.
<code>comp_use/cli.py</code>	828 lines. The orchestration facade. Browser session lifetime as a context manager, the artifact store with its approve/reject/retire/default lifecycle, version allocation under a process-wide lock, success-checkpoint derivation, per-run allowlist widening.

<code>comp_use/server/app.py</code>	FastAPI. Capability CRUD, invoke and discover returning 202 with a run id to poll, run control, version lifecycle, chat. Validates capability names and run ids against a strict pattern before they ever reach a filesystem path.
<code>comp_use/server/run_manager.py</code>	One background thread per run, with a lifecycle state machine: running → escalated → done / error / interrupted. Publishes the transport into the record before the record becomes visible, so a concurrent control request can't observe a half-built record.
<code>comp_use/chat/{agent,session,catalog}.py</code>	The conversational client. Tool definitions, the deterministic confirmation pre-parse, per-session locking and tokenization, and the catalog builder that scopes the model's search space to approved capabilities on the selected site.

Patterns and keywords

Named patterns you can drop in conversation, with where they actually appear.

PATTERN	WHERE	SAY IT AS
Ports & Adapters (Hexagonal)	<code>surface.py</code> , <code>llm_client.py</code> , <code>transport.py</code>	"Three abstract interfaces with swappable implementations — browser, model provider, human transport. None of the engine code knows which it's talking to."
Record & Replay	The whole architecture	The thesis. Also called <i>trace-based</i> or <i>demonstration-based</i> automation.
Interpreter	<code>replay/engine.py</code>	"The artifact is a program; the engine is its interpreter."
Compiler pass	<code>discovery/compiler.py</code>	Trace to artifact, with dedup and dead-extract-elimination passes.
Chain of Responsibility	Locator <code>fallback</code>	Recursive fallback with a short probe timeout for every non-terminal attempt.
Circuit breaker	Loop and dead-end detection	Bounded attempts before escalating to a human.
Registry	<code>outcome_library.py</code>	Host-keyed lookup, resolved at runtime, empty for unknown hosts.
Decorator	<code>FallbackLLMClient</code>	Wraps a client, transparently falls through to a second provider.
Template Method	<code>_OpenAICompatibleClient</code>	Shared request flow; subclasses override endpoint, headers, model names.
Facade	<code>cli.py</code>	One entry point that both the CLI and the HTTP server call.
Test double	<code>FakeLLMClient</code>	Why 368 tests run without a network or an API key.
Async job + polling	<code>app.py</code> + <code>run_manager.py</code>	<code>202 Accepted</code> with a run id; the client polls. Browser runs outlive an HTTP request.
Fail-closed	Chat confirmation, risky steps	Every ambiguous path resolves to the safe outcome.

PATTERN	WHERE	SAY IT AS
Defence in depth	Allowlist / redact / tokenize	Three independent layers, one shared config.
Dual control (four-eyes)	Risk tiers + approval lifecycle	Native banking vocabulary — use it. A human confirms risky steps <i>and</i> a human approves the artifact before it can ever be invoked.
Immutable versioning	Artifact store	Never overwrite; bump. Re-recording produces a draft while the approved version keeps serving.
Graceful degradation	Postgres, screenshots, derived outputs	Every non-load-bearing subsystem fails soft.

Questions to expect

Prepared answers. The first four are near-certain.

What happens when the site changes?

Three layers. The locator fallback chain handles a control findable a second way. Drift diagnosis handles a control that moved or got renamed — it proposes a patched version and escalates for human review, and nothing is auto-applied. Beyond that, the capability fails cleanly as a hard failure naming the step and what it expected, and you re-record. Be explicit that auto-applying a self-healed selector against a ledger is a deliberate non-goal.

Why not just use their API?

Because the premise is systems that don't have one. This targets a legacy core-banking web app with table-based layouts and no label semantics. Where an API exists, use it — this is for the tier below that, where today the alternative is a human clicking through a green screen.

How do we point this at our system?

Add an entry to the outcome registry for your host's error and denial pages, and set the allowlist. Then record capabilities against it. The registry is the only host-specific file; both engines are untouched. Recording a capability is one ~100-second session plus a human review.

What does it cost at scale?

Recording: ~71k tokens once per capability. Invoking: zero tokens, forever. Compute: budget ~450 MiB per concurrent run, 1.2 GiB worst case, one ephemeral container each, nothing left behind. The economics are the argument for the split — cost is per *capability*, not per *call*.

You're using free-tier models — is that serious?

The provider is a config line. The architecture is deliberately provider-agnostic and there's a fallback wrapper that fails over between two. For a bank the target is an on-prem or VPC-hosted model, and the tokenization boundary means most of the pipeline never sends real values anywhere regardless. Volunteer that the vision fallback is the exception.

How do you know a recorded capability is correct?

Three ways, and say all three. Discovery refuses to finish on several classes of incomplete recording. The artifact is a JSON document a human reviews and explicitly approves before it can ever be invoked — an unapproved draft is not invocable. And the artifact carries its own success checkpoint, so every replay proves it reached the intended end state rather than assuming it.

What about concurrency and races?

Version allocation is under a process-wide lock, never held across a blocking call. Run ids carry a uuid suffix because a millisecond timestamp collides under real load. The run manager refuses to overwrite a live record rather than silently evicting it. Chat sessions hold a per-session lock so two overlapping requests can't both confirm the same pending action. Each of those was a real bug.

Test coverage?

368 tests, no network required — the LLM client has a fake, and the mock app exercises the browser paths. The heaviest suites are the two engines: ~1,400 lines testing discovery, ~1,270 testing replay.

What would you do differently?

Have an answer ready. Candidates: model session-timeout recovery as a first-class re-authenticate-and-resume rather than a single recovery step; persist chat sessions; strip the desktop environment from the sandbox image; and make the outcome registry data rather than code, so onboarding a host is a config change instead of a deploy.